

GDEV 30 Programming Exercise 3 - Transformations and Coordinate Spaces

Specifications

In this exercise, you will be using the GLM library to compute transformation matrices, and then apply those matrices to the vertices of a 3D model that you shall create from scratch.

Preparation

1. Starting from the original demo1.cpp, redefine the vertex array so that it renders a simple square, as follows:

```
float vertices[] =
{
    // position (x, y, z)    color (r, g, b)
    -0.50f, -0.50f, -1.00f,  1.0f, 1.0f, 1.0f,
     0.50f, -0.50f, -1.00f,  1.0f, 1.0f, 1.0f,
    -0.50f,  0.50f, -1.00f,  1.0f, 1.0f, 1.0f,

     0.50f,  0.50f, -1.00f,  1.0f, 1.0f, 1.0f,
    -0.50f,  0.50f, -1.00f,  1.0f, 1.0f, 1.0f,
     0.50f, -0.50f, -1.00f,  1.0f, 1.0f, 1.0f
};
```

Note that format of the vertices above implies that you have two vertex attribute pointers in your VAO – one for position (3 floats) and one for color (also 3 floats). Adjust your VAO setup code as follows:

```
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 6 * sizeof(float),
                      (void*) 0);
glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 6 * sizeof(float),
                      (void*) (3 * sizeof(float)));
glEnableVertexAttribArray(0);
glEnableVertexAttribArray(1);
```

2. If you try running the program right now, this “square” will actually display as a rectangle. That’s because we are currently rendering using *normalized device coordinates* on a non-square window – that is, we have not properly set up the scene for 3D viewing yet.

To set up the scene for 3D viewing, we will need an initial transformation matrix called a *projection transformation*. In your `render()` function, right after the `glUseProgram(shader);` line, insert the following code:

```
glEnable(GL_DEPTH_TEST); // enable OpenGL's hidden surface removal

glm::mat4 matrix;
matrix = glm::perspective(glm::radians(60.0f),
                          (float) WINDOW_WIDTH / WINDOW_HEIGHT,
                          0.1f,
                          100.0f);

glUniformMatrix4fv(glGetUniformLocation(shader, "matrix"),
                  1, GL_FALSE, glm::value_ptr(matrix));
```

This sets up a *perspective projection* with a field-of-view (FOV) angle of 60 degrees. (This will be explained in the Coordinate Spaces slides, but for now just use this as-is.)

3. The above code will not have any noticeable effect yet, because you still need to process the matrix in your GLSL shader. To process the matrix, change your vertex shader (.vs) to the following:

```
#version 330 core

layout (location = 0) in vec3 vertexPosition;
layout (location = 1) in vec3 vertexColor;

uniform mat4 matrix;
out vec3 shaderColor;

void main()
{
    gl_Position = matrix * vec4(vertexPosition, 1.0f);
    shaderColor = vertexColor;
}
```

Also change your fragment shader (.fs) to the following:

```
#version 330 core

in vec3 shaderColor;
out vec4 fragmentColor;

void main()
{
    fragmentColor = vec4(shaderColor, 1.0f);
}
```

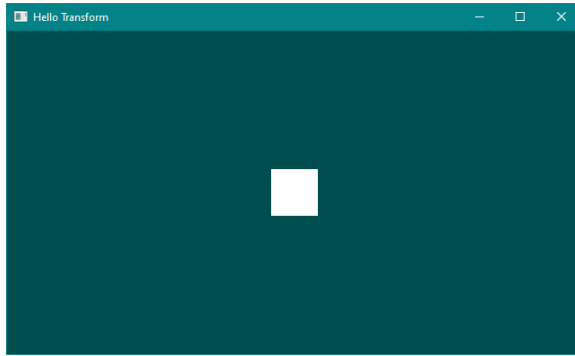
If you did this correctly, you should now see a proper square:



4. Now you can experiment with different kinds of affine transformations! For example, adding the following C++ code between the `glm::perspective` and `glUniformMatrix4fv` calls:

```
matrix = glm::translate(matrix, glm::vec3(0.0f, 0.0f, -5.0f));
```

will result in the square appearing to “shrink” (in reality, it is simply moved away from the origin; it appears smaller due to perspective foreshortening):



If you add the following code immediately after the `glm::translate`:

```
matrix = glm::rotate(matrix, glm::radians(-90.0f),
                      glm::vec3(1.0f, 0.0f, 0.0f));
matrix = glm::scale(matrix, glm::vec3(5.0f, 5.0f, 1.0f));
```

it will result in the square rotated so that it is lying down (facing upwards), and scaled to five times its size in the square's own local X and Y axes:



Now, let's discuss the real exercise!

Tasks

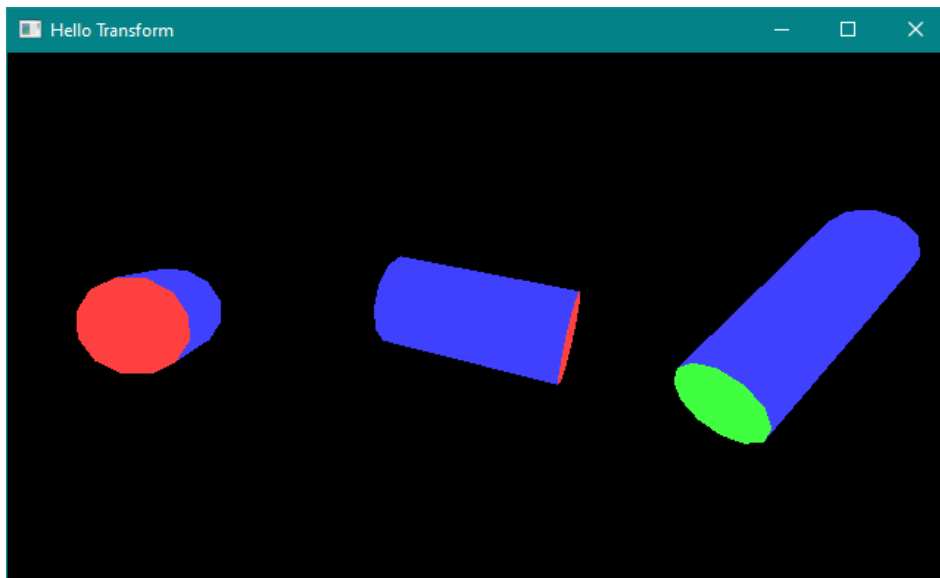
1. (60 points) Define a vertex array for a 3D model with **at least 30 triangles**.
 - Possible ideas for your 3D model:
 - A classical geometric 3D object (such as a cylinder, cone, sphere, torus, etc.), subdivided using triangles or quadrilaterals. (Don't just make a cube!)
 - A simple weapon model for a game (such as a laser gun, axe, staff, etc.)
 - Any other object you can think of with at least 30 triangles (spaceship, ice cream cone, gingerbread man, teapot...)
 - Make sure the center of your model is at (0, 0, 0), and please do not exceed the range (-1, -1, -1) to (1, 1, 1) when specifying your vertices.
 - Specify vertex colors! If you specify one color throughout, your model will look flat.
 - You will be graded not just on the complexity of your model, but on how structurally solid it is – your model must be closed or “airtight”, and not have triangles cutting through each other.
 - While modeling, you may want to use the `glm::translate` command as shown above (so you can see the object pushed back from the world origin), and use `glm::rotate` to see it from different angles (don't use `glm::scale` for now).

- As much as possible, specify all triangles so that they are winding in **counter-clockwise order** for each triangle facing the viewer, and the opposite way for each triangle facing away. (This is in preparation for the next exercise, where you will be using this model again!)
- An easy way to check for proper counter-clockwise winding order is to use the OpenGL “face culling” feature. With the following command, OpenGL will not render any triangle that is facing away from the viewer (i.e., whose coordinates map to screen space in a clockwise manner):

```
glEnable(GL_CULL_FACE);
```

2. (20 points) Draw **at least 3 copies** of your model on the screen, each with different **translation, rotation, and scaling** transformations. To get full credit for this, you should have each copy independently **rotating on different axes over time**, using the value of (float) glfwGetTime().
3. (5 points) Use the `glm::lookAt()` function to place a **virtual camera** on your scene (instead of the viewer being centered at the world’s origin). Details on how to use the `glm::lookAt()` function will be revealed in the Coordinate Spaces slides. Feel free to specify the parameters of your virtual camera, but all copies of your model must be visible!

Here’s an example of an acceptable camera view. In this example, the model is a blue cylinder with a red top and a green base, and each model instance is rotated and scaled differently:



4. (15 points) If you complete the above requirements perfectly, you will only get a B for this exercise. To get a B+ or A, you will need to implement BOTH of the following:
 - a. **Textured model**. Load a texture file, and specify (s, t) coordinates in your vertex array. (Don’t forget to modify the VAO’s attributes!) You may use more than one texture and apply any effects that you want – the nicer your texturing/effects are, the higher your grade will be.
 - b. **Controls for the virtual camera**. Feel free to specify your control mechanics (but please mention what your controls are in the code comments at the top of your .cpp file). For example, you can simply make the camera orbit around the scene

using the arrow keys, or you can provide first-person shooter controls using WASD and mouse or arrow keys. The more controllable your camera is, the higher your grade will be.

Submission

1. Your submission should contain:
 - A single .cpp file.
 - Any number of support files – either shader files (.vs/.fs) or texture files (.jpg/.png).
 - A filled-in Certificate of Authorship as a .pdf.
2. Your program should expect all support files from the same folder as the executable (i.e., do not use subfolders).
3. Do **not** include an .exe or .out file in your submission. I will compile the .cpp myself.
4. Do **not** include the libraries (e.g., GLAD, GLFW, GLM, stb_image.h, and gdev.h) in your submission. I do not expect you to use any additional libraries for this exercise.
5. Archive all your files in a single **.zip** file (do not use other formats like .rar, .7z, or .tar.gz) and submit the .zip file via Canvas.